

銷售提案 Sales Kit ・通用版

xConsole
模型主控台

統一記憶體 AI 機的模型主控台：載入前先試算，網頁上換模型，同仁的 IDE 一頁串好

云碩科技股份有限公司　2026 年 10 月

AI 機買回來了，接下來每一天都要有人顧著它

「再多開一個模型，應該還放得下吧？」

AMD AI 395 、 Jetson 、 DGX Spark 這類統一記憶體的 AI 機， CPU 和 GPU 共用同一塊記憶體。開一個模型還好，開到第二、第三個，或是同事一次送進四萬字的程式碼，記憶體就悄悄見底。它不會跳出錯誤，而是整台越來越慢，最後連 SSH 都進不去，只能走到機器前面按電源。

平常要換模型、查哪台開了什麼、告訴同事 IDE 要填哪個網址，也都得找那一兩位會用指令的工程師。機器一多，這些事就變成一張只有他看得懂的筆記。問題不在人，在缺一個把這些事收進網頁的主控台。

真實發生過：我們自己的 AI 機被吃到卡住

云碩自有機群（兩台 AMD AI 395 Strix Halo ）的維運紀錄，這也是我們做 xConsole 的起點

1 日常

同一台 AI 395 上，寫程式的模型給同事用，抓資料的清洗模型同時在跑

2 10 月 2 日

系統記憶體只剩 1 GiB 可用，置換空間用掉 25 GiB ，每段要跑 67 秒

3 更早以前

記憶體被吃爆時不只一次整台卡死， SSH 進不去，只能到機房按電源重開

4 10 月 20 日

裝上 xConsole 後，同一台想再載一顆 64 GiB 模型，試算剩 3.2 GiB ，擋下

統一記憶體機被吃爆時不會跳錯誤，而是整台越來越慢；等到發現，往往已經連不進去。要防的不是「現在用了多少」，是「載下去之後最壞剩多少」。

資料來源：云碩自有機群維運紀錄（ 2026 年 10 月 2 日記憶體量測）、 xConsole 准入試算紀錄（ 2026 年 10 月 20 日）。

記憶體不夠，先擋下來

不是等整台當掉才知道

先算再載入、網頁上換模型、同仁自己串 IDE ，主管每月看得到用量

一頁看懂：誰在用、能做什麼、跑在哪裡

同仁只接觸最上面一層；所有能力跑在單位自己的 AI 機上



差在哪：以前靠人記，現在交給主控台

下方數字為 2026 年 10 月 22 日實際執行測試的結果

以前：靠一兩位會指令的工程師

開下去才知道夠不夠

吃爆就整台卡死，到機房重開

換模型要連進機器

改設定檔、重啟，失敗就停擺

狀態記在個人筆記

哪台跑什麼、網址多少要問人

xConsole：一個網頁收起來

載入前先試算

最壞剩多少不夠，就擋下來

網頁上換模型

起不來自動換回原本那顆

串接資訊一頁看完

範例直接複製，按一下實測

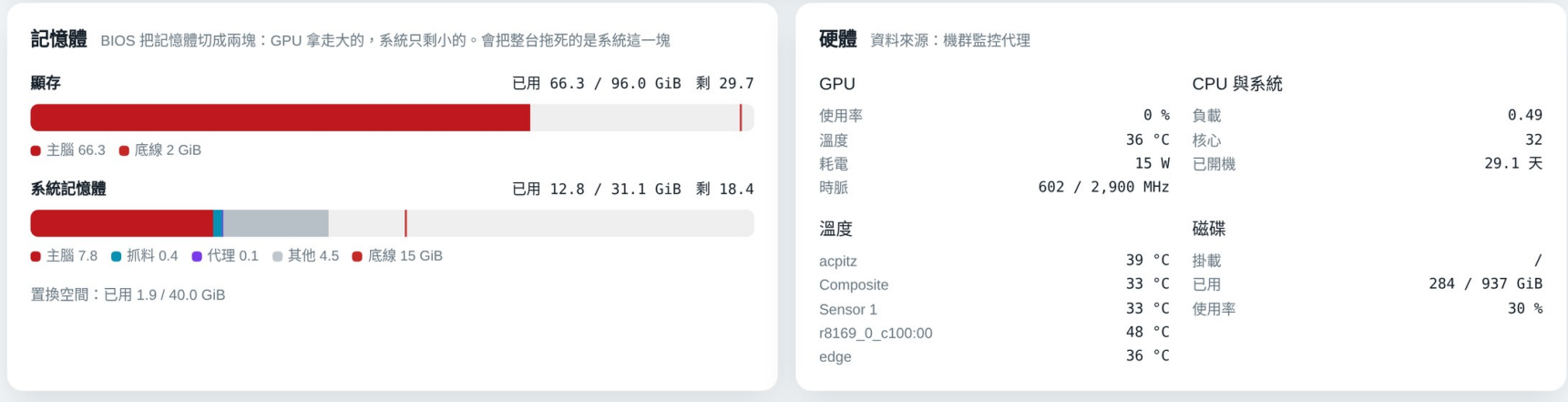
214 項	52 項	3 種	2 種
自動化測試全數通過	攻擊式安全測試全數通過	記憶體架構分開算	推論引擎同一個畫面

記憶體准入：載下去之前，先算最壞剩多少

01 · 記憶體准入控制

一個模型剛載入時用量不高，等到處理長提示才把記憶體撐開；只看當下用量，就會放行一個稍後會把機器拖垮的載入。

每次啟動、重啟、換模型、調大上下文之前，主控台先算載下去之後最壞剩多少，低於底線就擋，畫面直接說哪一塊不夠、差多少；正在跑的模型重啟時，會先把它自己占的記憶體算回來。



真實系統畫面（正式環境）：記憶體長條，紅線為安全底線

防的不是現在用了多少，是載下去之後剩多少。

三種記憶體架構，分開算才不會看錯數字

同一條公式套在不同機器上，會給出看起來安全、實際上危險的判斷

AMD AI 395 GPU與系統切兩塊	96 GiB給GPU 系統只剩31 GiB 會拖死整台的是系統這塊 系統底線15 GiB	切兩塊
Jetson / DGX 單一池子共用	GPU與系統搶同一塊 合併計算，不分開說都夠 Thor 122 GiB 底線12 GiB	單一池
多卡伺服器 每張卡各自算	每張顯示卡各自的顯存 系統記憶體另外算 顯存底線1 GiB 系統底線8 GiB	多卡
每次都算 啟動、換模型前	載下去之後最壞剩多少，低於底線就擋，並說明差多少	試算

網頁上換模型：選一顆，起不來就自動換回

02 ・換模型

換模型以前要連進機器改設定檔再重啟；改錯一個參數，引擎幾秒就崩，模型就這樣停擺。

換模型視窗同時列出兩種引擎的模型： Qwen3.8 家族走 Strata ，其他 GGUF 模型（ Gemma 、 gpt-oss 、 Llama ）走 llama.cpp 。套用時先備份、先試算，再重啟並實際打一次模型清單；打不通就整組換回原本的引擎與模型。跨引擎時同一個網址換人接手。

換模型是點一下的事，失敗也不會留下一台停擺的機器。



真實系統畫面（正式環境）：換模型視窗，兩種引擎一起列

模型庫：貼上名稱就下載，下載完就能換

03 · 模型庫

想試一顆新模型，要知道檔案放哪個資料夾、名稱要怎麼對、磁碟夠不夠；放錯位置，換模型清單裡就看不到。

每台機器一個固定的存放區。在模型庫貼上 Hugging Face 名稱、選量化就下載，下載前先查磁碟，中斷了會接續；推測解碼用的輔助檔會標出來，中國來源的模型也會標示。下載完立刻出現在換模型清單。

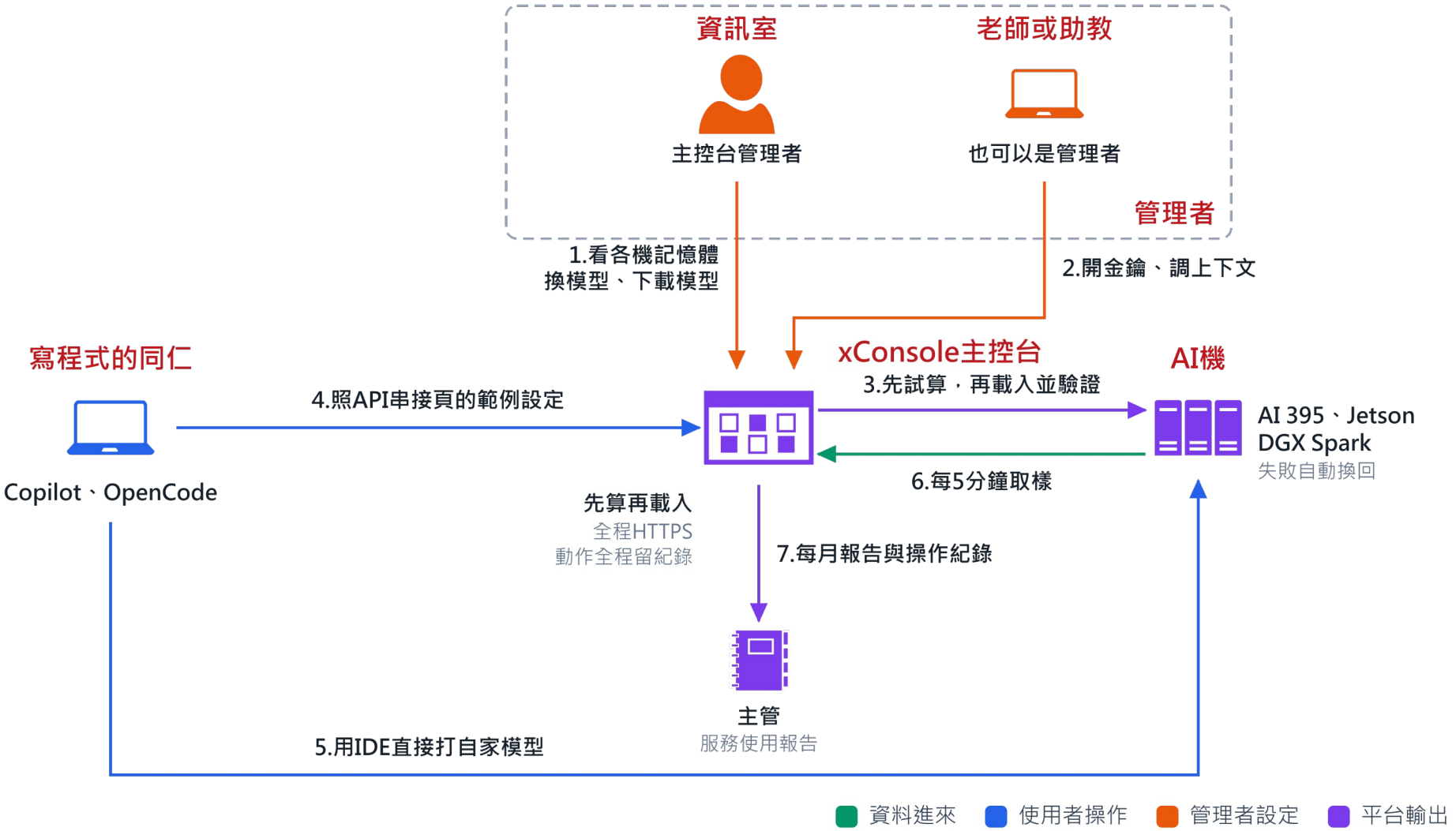


真實系統畫面（正式環境）：模型庫的存放區與 Hugging Face 下載

放模型不必懂資料夾規則，下載完就換得過去。

使用者操作流程：從換模型到月報，七個動作

管理者在網頁上操作；同仁只要照設定範例接上；主管拿到的是報告



同仁的 IDE 一頁串好：網址、名稱、上下文、金鑰都在這裡

04 · API 串接

換完模型，同仁問的永遠是同一件事：網址是什麼、模型名稱要填什麼、上下文可以開多長；工具一多，每個人的設定都不一樣。

API 串接頁把這些放在同一頁，也能在網頁上改；依目前設定產出 8 種工具的設定範例，按一下實測，從本機與經網路各打一次，連工具呼叫一起驗。

同仁照範例貼上，按一次實測，就知道接不接得通。



真實系統畫面（正式環境）：實測一次，對話約 1.5 秒、工具呼叫約 2.3 秒

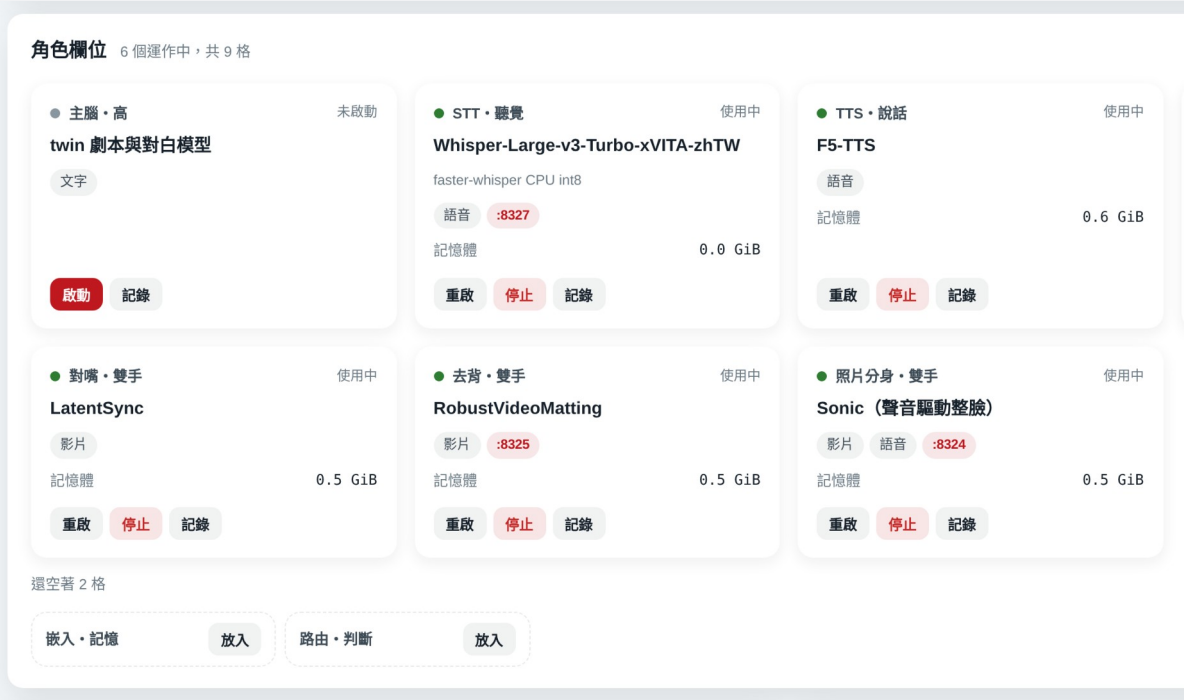
角色欄位：每台機器缺什麼，一眼看得出來

05 ・角色欄位

機器上開了哪些服務，以前要連進去一個一個查；哪個是同事在用的關鍵服務，只有負責的人知道。

每台機器以角色排列：主腦、聽覺、說話、記憶、判斷、雙手、眼睛各一格，空著的直接顯示。每格顯示模型、能力、埠號、記憶體與實測速度；關鍵服務停止前多一道確認，並說明停掉會影響誰。

空著的格子就是這台還缺的能力。



真實系統畫面（正式環境）： Jetson AGX Thor 的角色欄位

憑證管理者自己換：上傳、檢查、套用，不必重開

06 · HTTPS 與憑證

憑證每年要換，以前要請工程師進主機；私鑰不成對、少了中繼憑證，整站就連不上，常常就在憑證過期的那個早上。

第一次開機就是 HTTPS 。管理者在網頁上傳 PFX 、 PEM 或 P7B ，系統逐項用白話檢查，套用時不重開服務，失敗自動換回；也能直接產生 CSR 交給憑證廠商。

換憑證變成管理者十分鐘的工作，不必叫工程師。



真實系統畫面（正式環境）：憑證設定順序軸

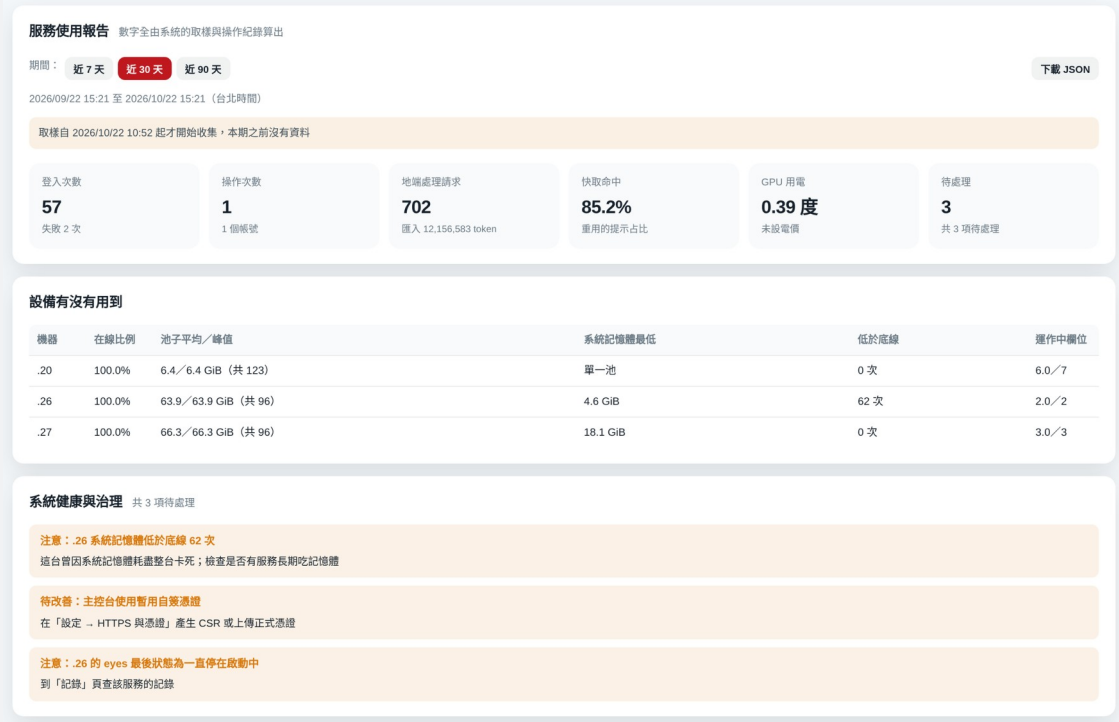
給主管看的：一份報告講清楚誰在用、用了多少

07 ・服務使用報告

主管問這幾台 AI 機到底有沒有人在用、值不值得再買，答案散在各台機器的紀錄裡，整理一次要半天。

主控台每 5 分鐘記錄各機狀態。報告列出登入與操作次數、 GPU 耗電、請求數與 token 量、各機記憶體峰值與低於底線的次數，連主控台自己的問題都會列成待處理事項；一行指令產出 Word 月報。

報告裡的數字是主控台算的，不是人填的。



真實系統畫面（正式環境）：服務使用報告

不是說說而已：實測數字與交付證據

數字來自 2026 年 10 月 22 日實際執行的測試

214 項

自動化測試全數通過

記憶體試算、換模型與還原、 API 串接、憑證、報告與安全測試

52 項

攻擊式安全測試全數通過

未登入存取、竄改登入、越權、路徑穿越、暴力猜測等

12 項

真機冒煙測試全數通過

真的 Hugging Face 下載加真的 llama.cpp ，跑完即刪

交付證據包

安全計畫、威脅模型、 NIST SSDF 對照表、 OWASP ASVS L2 檢核表、 OWASP Top 10 對照表、 CycloneDX SBOM 、弱點掃描報告與自測紀錄。

每次修改都由 CI 重跑測試與六道安全掃描，全部通過才能合併；尚未做到的項目照實列在缺口清單並訂有期限。

交付時的安全做法：每一項都對得到證據

以資通系統防護基準中級、 OWASP ASVS L2 為基準

項目	做法	證據
連線加密	第一次開機即 HTTPS ， TLS 1.2 以上， http 一律轉走	安全測試、 HTTPS 檢查
登入	伺服器端登入票證，登出與換密碼即作廢； 15 分鐘錯 5 次即擋	安全測試第 2 、 9 節
機密	機器間權杖 AES-256-GCM 加密存放；模型金鑰只顯示一次	機密稽核、安全測試
操作紀錄	誰、何時、從哪裡、做了什麼；寫入前遮罩，保存 400 天	ASVS 檢核表 V16
輸入與路徑	只接受掃描清單內的模型與檔名，路徑一律限定在目錄內	CodeQL 、安全測試
供應鏈	依賴釘版本， SBOM 與弱點掃描， CI 全綠才能合併	SBOM 、掃描報告
模型來源	中國來源的模型在模型庫自動標示	模型庫畫面
事件應變	知悉後 1 小時內配合機關通報	安全計畫第 5 節

資訊單位最在意的三件事：不當機、不靠特定人、說得出用量

依單位類型交付

AI 機進到單位裡，資訊單位要面對的不是模型準不準，而是三件很實際的事：機器會不會又卡死要人跑機房；會操作的工程師請假或離職時，其他人接不接得起來；主管問這幾台機器到底有沒有人在用、值不值得再買，拿不拿得出數字。

xConsole 把載入前的記憶體試算做成每次啟動、換模型都會先跑的關卡，放不下就擋，並用白話說明差多少。換模型、下載模型、調上下文、開金鑰全部在網頁上完成，每個動作都寫進操作紀錄；系統每 5 分鐘記錄各機狀態，一行指令產出給主管看的服務使用報告，裡面連平台自己的問題都會列出來。

載入前先試算

網頁上操作

每月使用報告

機器顧得住、操作交得出去、用量說得清楚。

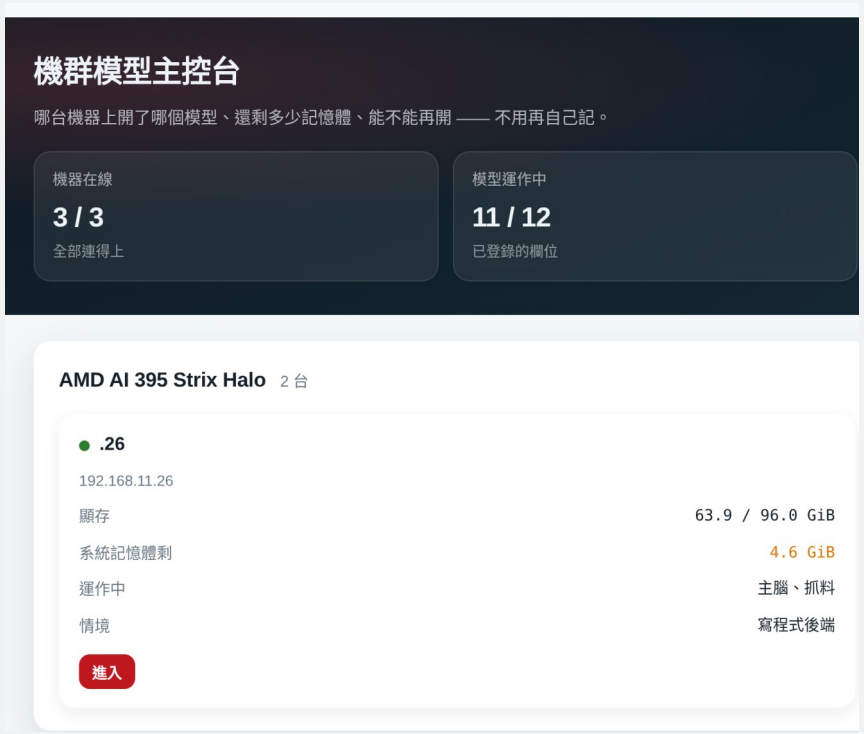
一台也管，十台也管：機器變多，設定不用重做

08 · 部署方式

AI 機一台一台買進來，每台各自的設定、各自的筆記；第三台開始，就沒有人說得清全部的狀況。

單機時，主控台直接裝在那台 AI 機上，開瀏覽器就能用。機器變多時，選一台當主控台，其他機器各裝節點代理；各機的設定不用重做，總覽頁一次看完每台的記憶體與運作中的角色，主控台連各機也走 HTTPS 並驗證對方。

從第一台到第十台，用的是同一個主控台。



真實系統畫面（正式環境）：總覽頁，同時管兩台 AI 395 與一台 Jetson

導入前後：四個改變

左邊是今天的做法，右邊是導入 xConsole 之後

	導入前		導入後
載入模型	開下去才知道夠不夠，吃爆就整台卡死	→	**載入前先算最壞剩多少，放不下就擋**
換模型	找工程師連進機器改設定檔再重啟	→	**網頁上選，起不來自動換回原本那顆**
IDE 串接	網址、模型名稱、上下文到處問	→	**一頁看完，範例直接複製，按一下實測**
主管報告	機器有沒有在用，說不清楚	→	**每月報告：用量、耗電、低於底線的次數**

為什麼是云碩

我們自己每天在用，出問題時改程式的是同一批人

自己每天在用	兩台 AMD AI 395 與一台 Jetson AGX Thor 每天由 xConsole 管著，寫程式的模型與抓資料的引擎都跑在上面。
自有機群經驗	自有 GPU 機群橫跨 NVIDIA 與 AMD ，涵蓋 AI 395 、 Jetson 、 DGX Spark 與多卡伺服器，知道統一記憶體機實際會出什麼問題。
平台自己寫	主控台與節點代理都是自家開發，客戶要改要加，我們動得了原始碼。
照合規流程交付	依 SSDLC 流程交付安全計畫、威脅模型、 SSDF / ASVS / Top 10 對照表、 SBOM 與掃描報告。

導入流程：四個階段，每階段都有交付物

全程在單位內網；由云碩工程師到場或依貴單位規定協助

1 裝上主控台

單機直接附在 AI 機上；多台時選一台當主控台，第一次開機就是 HTTPS 。

2 納管機器

每台裝節點代理，登錄角色欄位與記憶體底線，主控台經 HTTPS 驗證每台。

3 放模型、接 IDE

下載或放入模型，在網頁上換好，把 API 串接頁的設定交給同仁。

4 換憑證、出月報

換上單位正式憑證，產出第一份服務使用報告。

開工前請貴單位確認：要納管的 AI 機與網段、正式憑證或內部 CA 、要給同仁使用的模型、主控台管理者。

聯絡窗口　**云碩科技股份有限公司**　台北．台中．高雄．新竹



Thank You

xConsole 模型主控台　統一記憶體 AI 機的模型編組與記憶體准入控制

云碩科技股份有限公司　xCloudinfo Corp. Limited

自主 AI ・地端部署・資料不離企業內網